



## **Chapter S1F: Linked List**

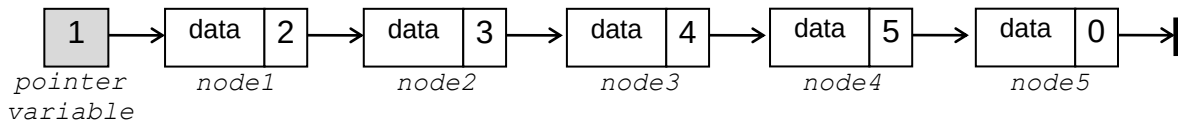
### **CONTENT**

| <b>No.</b> | <b>Topic</b>                                                                                                                 | <b>Syllabus Guide</b> |
|------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| 1          | Linked List                                                                                                                  | 1.3.5                 |
| 2          | Abstract Data Type                                                                                                           |                       |
| 3          | Linked List Traversal                                                                                                        |                       |
| 4          | Free Space List                                                                                                              | 1.3.4                 |
| 5          | Implementation using arrays<br>5.1 Using a single array<br>5.2 Using two arrays                                              | 1.3.5                 |
| 6          | Implementation using an array of nodes                                                                                       |                       |
| 7          | Searching for a node                                                                                                         |                       |
| 8          | Inserting a node into a linked list                                                                                          |                       |
| 9          | Deleting a node from a linked list                                                                                           |                       |
| 10         | Ordered linked list<br>10.1 Inserting a node into an ordered linked list<br>10.2 Deleting a node from an ordered linked list |                       |
| 11         | Dynamic linked list                                                                                                          |                       |
| 12         | Comparing linked lists and arrays                                                                                            | 1.3.1, 1.3.2          |
| 13         | Tutorial                                                                                                                     |                       |
| 14         | Programming exercise                                                                                                         | 2.3.3                 |

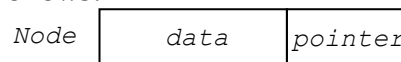
## 1. LINKED LIST

A linked list is a data structure consisting of a set of nodes to form a sequence. Each node consists of the **data value** and a **link pointer** to one of the other nodes. The purpose of using a linked list is to link the nodes in some particular order, such as alphabetical order of the data items.

Conceptually, a linked list may look like this:



Each node is represented as follows:



The nodes are not necessarily held in contiguous (physical) data locations, or in the order in which they occur in the sequence. A **pointer variable** points to (contains the address of) the *first node* in the linked list. The data field may consist of several sub-fields to hold more than one piece of data. The pointer field contains the address of the next node in the sequence. The pointer field of the last node indicates that there are no further nodes (e.g. has a value of 0, a null pointer).

## 2. ABSTRACT DATA TYPE (ADT) – LINKED LIST

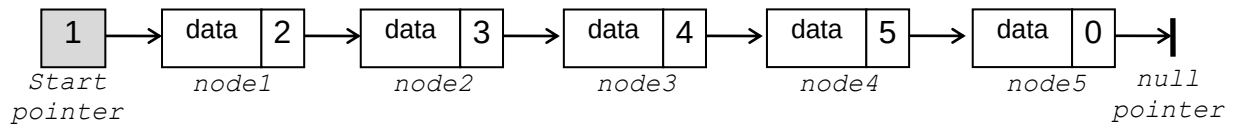
A linked list is an example of an Abstract Data Type (ADT). An ADT describes a data structure based on its behaviour from the point of view of a user, without specifying the implementation. It focuses on what the structure can do, not how it is implemented.

Linked List ADT supports the following core operations:

| Operation    | Description                                                              |
|--------------|--------------------------------------------------------------------------|
| Initialise   | Set up an empty linked list                                              |
| Insert(item) | Insert a new item into the linked list (e.g. at head, tail, or in order) |
| Delete(item) | Remove a specific item from the linked list                              |
| Search(item) | Look for an item and return its position or reference                    |
| Traverse()   | Visit and optionally display all elements from start to end              |
| IsEmpty()    | Check whether the list has no elements                                   |

### 3. LINKED LIST TRAVERSAL

To traverse a linked list, you follow the pointer chain from the Start node to the end of the list (null pointer). This process is used in searching, inserting, deleting or counting items.



Pseudocode for traversal:

```

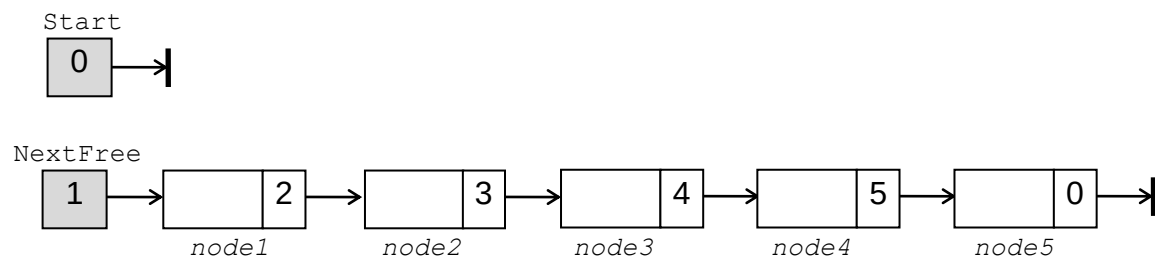
current ← Start
WHILE current ≠ NULL
    OUTPUT Data[current]
    current ← Pointer[current]
ENDWHILE
  
```

### 4. FREE SPACE LIST

A free space list operates by connecting unallocated regions of memory together in a linked list. This pool of memory contains objects of the same size, which are the nodes.

To manage the free space for a linked list data structure, two linked lists are kept: *one for the actual data*, and *one for the free nodes*. When a new item is added, a node is grabbed from the free space list, and when a node is deleted, it is linked into the free space list.

At the point of *initialisation*, there is just *one linked list of free nodes*. The linked list for the actual data is empty at this point.



| Node | Data | Pointer |
|------|------|---------|
| 1    |      | 2       |
| 2    |      | 3       |
| 3    |      | 4       |
| 4    |      | 5       |
| 5    |      | 0       |

```

Start = 0 (null)
NextFree = 1
  
```

## 5. IMPLEMENTATION USING ARRAYS

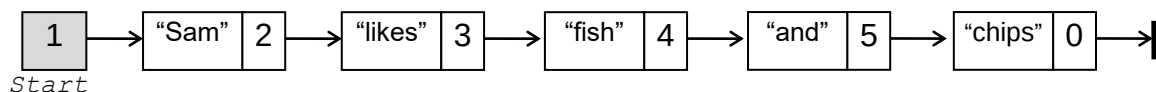
### 5.1 Using a single array

A linked list may be implemented using a single array with identifier *A*, where the total number of nodes is fixed, and two variables *Start* and *NextFree*. Each row represents a node. For example, *A*[5,1] (row 5, column 1) contains “chips”.

|                        |             |                          |                    |
|------------------------|-------------|--------------------------|--------------------|
| <b>Start</b><br>1 →    | <b>Node</b> | <b>Data</b>              | <b>Pointer</b>     |
|                        | 1           | <i>A</i> [1,1] = “Sam”   | <i>A</i> [1,2] = 2 |
|                        | 2           | <i>A</i> [2,1] = “likes” | <i>A</i> [2,2] = 3 |
|                        | 3           | <i>A</i> [3,1] = “fish”  | <i>A</i> [3,2] = 4 |
|                        | 4           | <i>A</i> [4,1] = “and”   | <i>A</i> [4,2] = 5 |
| <b>NextFree</b><br>6 → | 5           | <i>A</i> [5,1] = “chips” | <i>A</i> [5,2] = 0 |
|                        | 6           | <i>A</i> [6,1] =         | <i>A</i> [6,2] = 7 |
|                        | 7           | <i>A</i> [7,1] =         | <i>A</i> [7,2] = 8 |
|                        | 8           | <i>A</i> [8,1] =         | <i>A</i> [8,2] = 0 |

where 0 is null pointer

The conceptual linked list containing the actual data looks like this:



### 5.2 Using two arrays

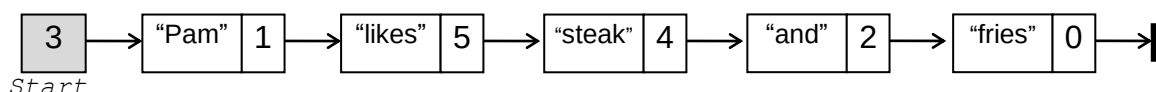
A linked list may also be implemented using 2 arrays and 2 variables as shown in this table.

| Identifier | Data type               | Description                                    |
|------------|-------------------------|------------------------------------------------|
| Data       | ARRAY[1..20] OF STRING  | The data value for the nodes                   |
| Link       | ARRAY[1..20] OF INTEGER | The link pointer value for the nodes           |
| Start      | INTEGER                 | The index of the first node of the linked list |
| NextFree   | INTEGER                 | The index of the next available free node      |

The data values are stored in the first array and the link values are stored in the second array:

|                        |     |             |     |             |
|------------------------|-----|-------------|-----|-------------|
|                        |     | <b>Data</b> |     | <b>Link</b> |
|                        | 1   | “likes”     | 1   | 5           |
|                        | 2   | “fries”     | 2   | 0           |
| <b>Start</b><br>3 →    | 3   | “Pam”       | 3   | 1           |
|                        | 4   | “and”       | 4   | 2           |
| <b>NextFree</b><br>6 → | 5   | “steak”     | 5   | 4           |
|                        | 6   |             | 6   |             |
|                        | ... |             | ... |             |
|                        | 20  |             | 20  |             |

The conceptual linked list containing the actual data looks like this:



## 6. IMPLEMENTATION USING AN ARRAY OF NODES

We can also define a linked list using user-defined types **Node** and **LinkedList** as follows:

| <b>Node</b>                  | <b>LinkedList</b>                              |
|------------------------------|------------------------------------------------|
| <b>Data:</b> STRING          | <b>NodesArray:</b> ARRAY[1..10] of <b>Node</b> |
| <b>Pointer:</b> INTEGER      | <b>Start:</b> INTEGER                          |
| Constructor()                | <b>NextFree:</b> INTEGER                       |
| GetData(): STRING            | Constructor()                                  |
| SetData(item: STRING)        | Insert(item: STRING)                           |
| GetPointer(): INTEGER        | Delete(item: STRING)                           |
| SetPointer(pointer: INTEGER) | Search(item: STRING)                           |
|                              | Display()                                      |

The linked list is implemented as an array of nodes, where the identifier of the array is **NodesArray**. This array contains 10 nodes, made up of user-defined type **Node**. The diagram shows 10 **Node** objects inside each of the array locations.

| <b>NodesArray</b> |      |     |    |  |          |   |
|-------------------|------|-----|----|--|----------|---|
| [1]               | Node | " " | 2  |  | Start    | 0 |
| [2]               | Node | " " | 3  |  | NextFree | 1 |
| [3]               | Node | " " | 4  |  |          |   |
| [4]               | Node | " " | 5  |  |          |   |
| [5]               | Node | " " | 6  |  |          |   |
| [6]               | Node | " " | 7  |  |          |   |
| [7]               | Node | " " | 8  |  |          |   |
| [8]               | Node | " " | 9  |  |          |   |
| [9]               | Node | " " | 10 |  |          |   |
| [10]              | Node | " " | 0  |  |          |   |

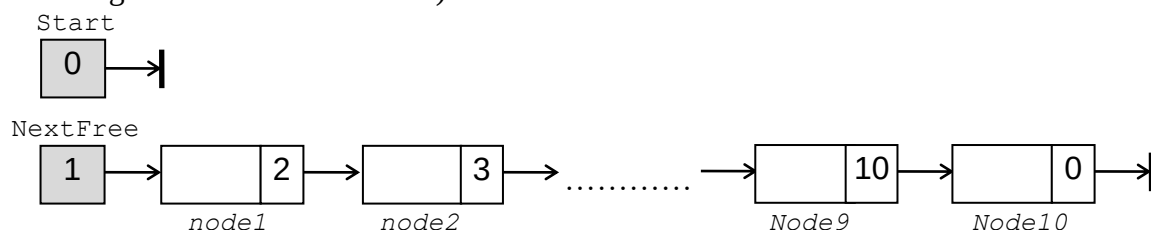
### Node type

| Attribute | Initial value                                                                                                                                             |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data      | Empty string " "                                                                                                                                          |
| Pointer   | All nodes (except last node) have next index value, e.g. pointer value of node in index 1 is initialised to 2. Last node pointer initialised to 0 (null). |

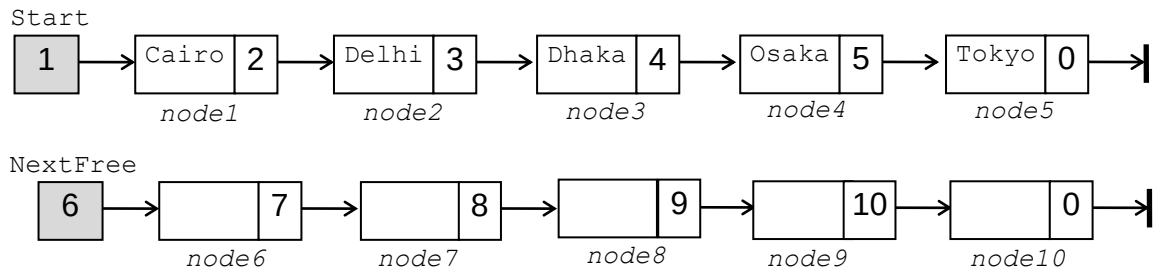
### LinkedList type

| Attribute | Initial value                              |
|-----------|--------------------------------------------|
| Start     | 0 (null value)                             |
| NextFree  | 1 (next available free node is at index 1) |

During initialisation, the linked list (that is to hold the actual data) and the **free space list** (containing the available free nodes) look like this:

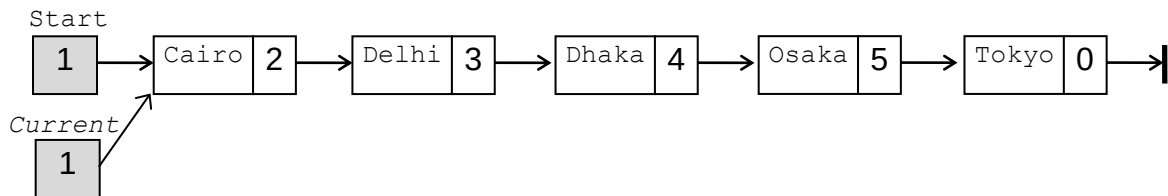


After some data are added into the linked list, the linked list (that is used to hold the actual data) and the **free space list** (containing the available free nodes) look like this:



## 7. SEARCHING FOR A NODE

- Step 1: Let **Current** = **Start** pointer and **data** = required data
- Step 2: If **Current** is **null** then print “not found” and exit
- Step 3: If content of present location = **data** then print **data** and exit
- Step 4: Let **Current** = next pointer of present location and go to step 2  
[present location is the location pointed to by **Current**]



If data required is “Tokyo”, trace through the algorithm to search for it.

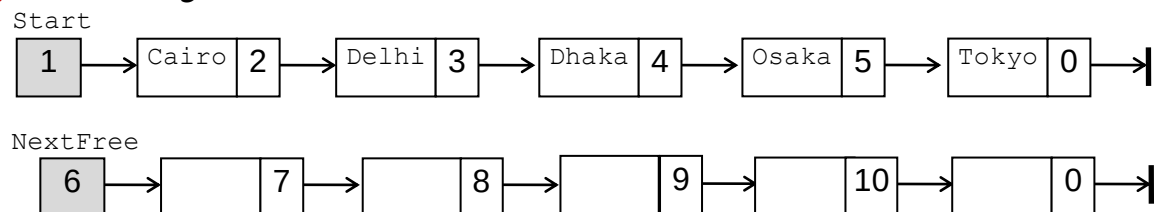
## 8. INSERTING A NODE INTO A LINKED LIST

Inserting an element into a linked list is achieved by *adjusting various pointers* to include the new element.

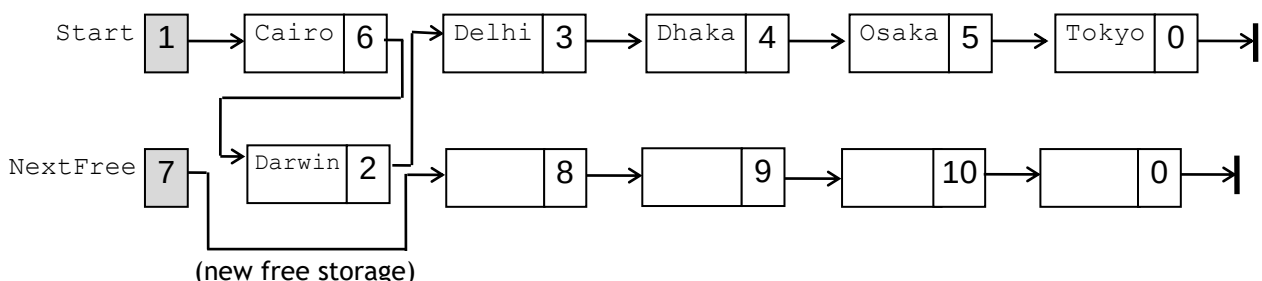
- Step 1: If free storage pointer **NextFree** is **null**, print “no more space” and exit.
- Step 2: **Search** for position into which new data is to be inserted. If failed, print “error message” and exit.
- Step 3: **Save** the location of the **first free space** and let the free storage pointer **NextFree** points to the next free space.
- Step 4: **Place** the **new data item** in the saved free space and let it point to the element **after** the **required position**.
- Step 5: **Change** the pointer of the element **before** the **required position** to point to the saved free element.
- Step 6: Exit.

The diagrams below show how “Darwin” is inserted at the 2nd position of the linked list.

**Before inserting node “Darwin”:**



**After inserting node “Darwin”:**



|          | Node | Data            | Pointer     |
|----------|------|-----------------|-------------|
| Start    | 1    | A[1,1] = Cairo  | A[1,2] = 6  |
|          | 2    | A[2,1] = Delhi  | A[2,2] = 3  |
|          | 3    | A[3,1] = Dhaka  | A[3,2] = 4  |
|          | 4    | A[4,1] = Osaka  | A[4,2] = 5  |
|          | 5    | A[5,1] = Tokyo  | A[5,2] = 0  |
| NextFree | 6    | A[6,1] = Darwin | A[6,2] = 2  |
|          | 7    | A[7,1] =        | A[7,2] = 8  |
|          | 8    | A[8,1] =        | A[8,2] = 9  |
|          | 9    | A[9,1] =        | A[9,2] = 10 |
|          | 10   | A[10,1] =       | A[10,2] = 0 |

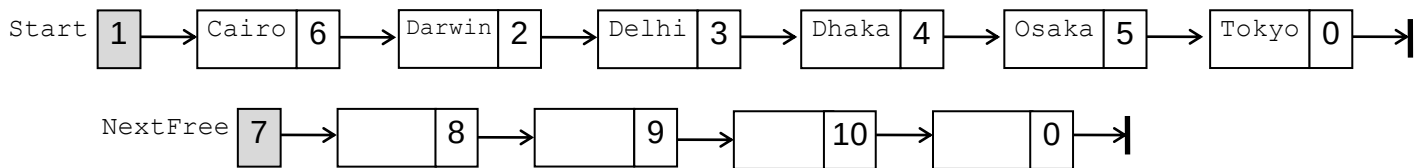
where 0 is null pointer

## 9. DELETING A NODE FROM A LINKED LIST

Deleting an element from a list can be similarly achieved as shown by the diagrams below. The free space created is returned to the head of the free space list.

- Step 1: **Search** for node containing data to be deleted. If not found, print “not found” and exit.  
 Step 2: **Save** location of node to be deleted by assigning a **Temp** pointer to it.  
 Step 3: **Change** pointer of node before **Temp** node to point to node after **Temp** node.  
 Step 4: **Change** pointer of **Temp** node to point to first free space.  
 Step 5: **Change** free storage pointer **NextFree** to point to **TEMP** node.  
 Step 6: Exit.

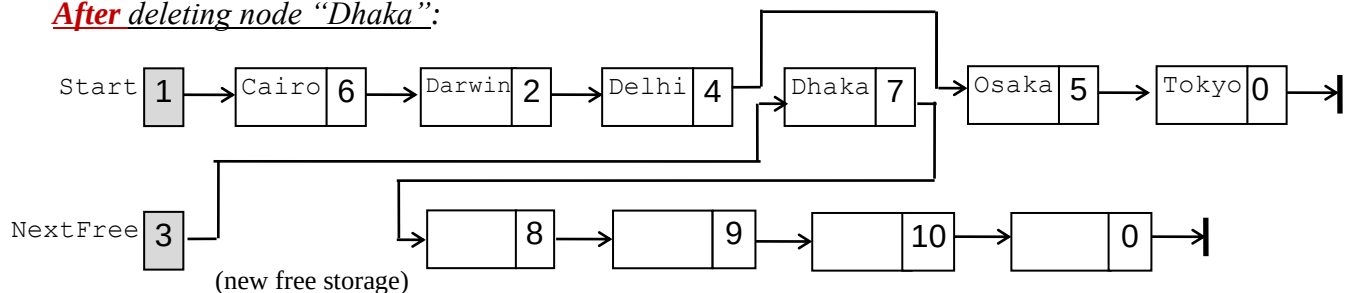
**Before** deleting node “Dhaka”:



| Node | Data            | Pointer     |
|------|-----------------|-------------|
| 1    | A[1,1] = Cairo  | A[1,2] = 6  |
| 2    | A[2,1] = Delhi  | A[2,2] = 3  |
| 3    | A[3,1] = Dhaka  | A[3,2] = 4  |
| 4    | A[4,1] = Osaka  | A[4,2] = 5  |
| 5    | A[5,1] = Tokyo  | A[5,2] = 0  |
| 6    | A[6,1] = Darwin | A[6,2] = 2  |
| 7    | A[7,1] =        | A[7,2] = 8  |
| 8    | A[8,1] =        | A[8,2] = 9  |
| 9    | A[9,1] =        | A[9,2] = 10 |
| 10   | A[10,1] =       | A[10,2] = 0 |

where 0 is null pointer

**After** deleting node “Dhaka”:

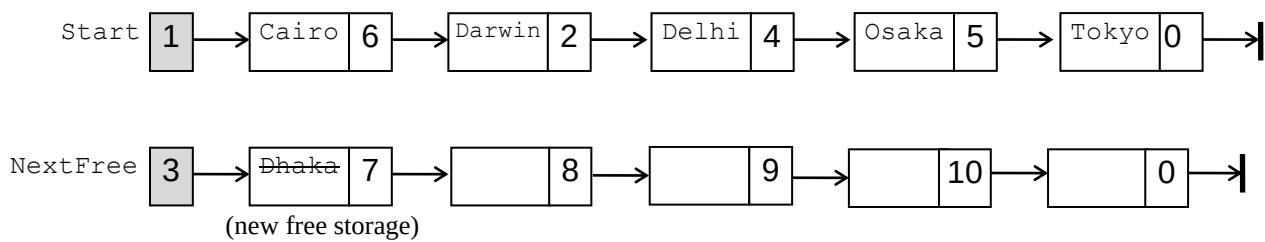


| Node | Data            | Pointer     |
|------|-----------------|-------------|
| 1    | A[1,1] = Cairo  | A[1,2] = 6  |
| 2    | A[2,1] = Delhi  | A[2,2] = 4  |
| 3    | A[3,1] = Dhaka  | A[3,2] = 7  |
| 4    | A[4,1] = Osaka  | A[4,2] = 5  |
| 5    | A[5,1] = Tokyo  | A[5,2] = 0  |
| 6    | A[6,1] = Darwin | A[6,2] = 2  |
| 7    | A[7,1] =        | A[7,2] = 8  |
| 8    | A[8,1] =        | A[8,2] = 9  |
| 9    | A[9,1] =        | A[9,2] = 10 |
| 10   | A[10,1] =       | A[10,2] = 0 |

where 0 is null pointer

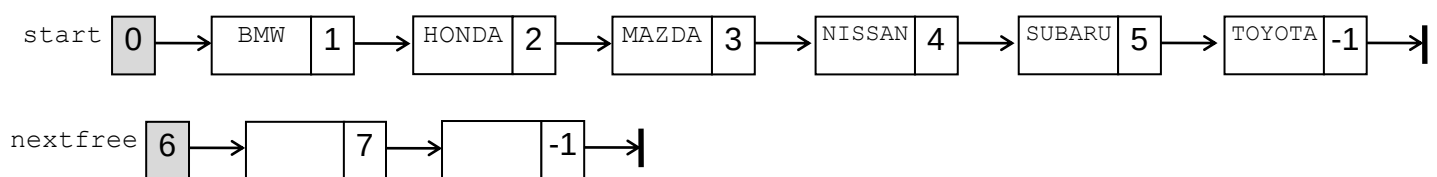


After realigning the linked list and the free space list:



## 10. ORDERED LINKED LIST

An ordered linked list stores data in some order, e.g. alphabetical or ascending.  
For example, this is a linked list of car make stored in alphabetical order:



**Make (an example)**

|              |              |             |                |
|--------------|--------------|-------------|----------------|
| start 0 →    | <b>index</b> | <b>name</b> | <b>pointer</b> |
|              | 0            | BMW         | 1              |
|              | 1            | HONDA       | 2              |
|              | 2            | MAZDA       | 3              |
|              | 3            | NISSAN      | 4              |
|              | 4            | SUBARU      | 5              |
|              | 5            | TOYOTA      | -1             |
| nextfree 6 → | 6            |             | 7              |
|              | 7            |             | -1             |

where -1 is null pointer

The linked list is implemented as an array of nodes, where the identifier of the array is **Make**.  
There are a total of 8 nodes in **Make**, and the index starts from 0 and ends at 7.  
The null pointer is indicated by -1.

### 10.1 Inserting a node into an ordered linked list

**Notation used:**

- **Make[index].name** holds the name in node[index]. For example, Make[2].name is MAZDA.
- **Make[index].pointer** holds the value of the pointer in node[index]. For example, Make[2].pointer is 3.

Referring to the table above, what is the value of the following?

- **Make[start].pointer**
- **Make[4].name**
- **Make[Make[3].pointer].pointer**
- **Make[Make[start].pointer].name**

Insert newData into “car make” linked list:

```

PROCEDURE Insert(newData)
  IF nextfree IS NULL THEN                                //check if free node exists
    output("No free node available") and RETURN
  ENDIF
  Make[nextfree].name ← newData                               //store newData in free node
  IF start IS NULL THEN                                     //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
  ELSE
    previous ← NULL
    current ← start
    WHILE current IS NOT NULL                               //traverse from start of linked list
      IF newData < Make[current].name THEN
        BREAK                                                 //Position found between previous and current node
      ENDIF
      previous ← current
      current ← Make[current].pointer
    ENDWHILE
    IF previous IS NULL THEN                                 // store as first node
      temp ← Make[nextfree].pointer
      Make[nextfree].pointer ← start
      start ← nextfree
      nextfree ← temp
    ELSE                                                       // store between previous and current
      temp ← Make[nextfree].pointer
      Make[nextfree].pointer ← current
      Make[previous].pointer ← nextfree
      nextfree ← temp
    ENDIF
  ENDIF
ENDPROCEDURE

```

Exercise: Draw diagrams to show insertion of newData into this linked list of “car make”.

## 10.2 Deleting a node from an ordered linked list

Delete Data from “car make” linked list:

```

PROCEDURE Delete(Data)
    IF start IS NULL THEN                                     //check if linked list is empty
        output("List is empty!") and RETURN
    ENDIF
    previous ← NULL
    current ← start
    WHILE current IS NOT NULL                                 //traverse from start of linked list
        IF Data = Make[current].name THEN                     //Data to be deleted is found
            read data and BREAK                               //current points to node to be deleted
        ENDIF
        previous ← current
        current ← Make[current].pointer
    ENDWHILE
    IF current IS NULL THEN                                 //reach end of linked list, node not found
        output("Node not found") and RETURN
    ENDIF
    IF previous IS NULL                                       //first node to be deleted
        temp ← nextfree
        nextfree ← current
        start ← Make[current].pointer
        Make[current].pointer ← temp
    ELSE                                                       //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
    ENDIF
ENDPROCEDURE

```

Exercise: Draw diagrams to show deletion of Data from this “car make” linked list.

## 11. DYNAMIC LINKED LIST

A dynamic data structure refers to a collection of data in memory that has the ability to grow or shrink in size during program execution. This is made possible by allocating and deallocating memory dynamically from the heap.

Python supports dynamic data structures such as its built-in list, which resizes automatically when elements are added or removed. However, the built-in list is actually a dynamic array, not a linked list. Many common operations like `append()`, `pop()`, and `remove()` are already implemented and can be used to construct abstract data types (ADT) such as stacks or queues.

A linked list, on the other hand, is inherently a dynamic data structure. It grows or shrinks by allocating or deallocating nodes as needed. In our earlier implementations using arrays, we simulated dynamic memory by managing a free space list. However, in real-world applications using object-oriented programming (e.g. in Python), there is no need to create a free space list manually. Memory is allocated when a new object is created and released when it is no longer referenced.

For example, to dynamically create a node in Python: `newNode=Node()`. This creates a new node object that can be added to the linked list. The Python runtime takes care of memory management, making linked list implementation simpler and more flexible.

## 12. COMPARING LINKED LISTS AND ARRAYS

| No. | Linked lists                                                                              | Arrays                                                                                     |
|-----|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1   | Nodes are not stored in contiguous memory; each node points to the next.                  | Elements are stored in contiguous memory locations.                                        |
| 2   | Dynamic in size – grows or shrinks during runtime.                                        | Fixed in size – must be declared in advance.                                               |
| 3   | Memory is allocated at runtime; no need to declare size.                                  | Memory is allocated at compile time (or at initialisation). Must know the size in advance. |
| 4   | Can grow by allocating memory in non-contiguous free space.                               | Resizing large arrays may require copying to new memory.                                   |
| 5   | Each node stores data and a pointer, using slightly more memory.                          | Stores only data, no per-element pointer overhead.                                         |
| 6   | Sequential access only – requires traversal ( $O(n)$ ). Does not support direct indexing. | Random access supported via indexing (e.g., <code>array[i]</code> ) in $O(1)$ time.        |
| 7   | Insertion/deletion is efficient at known positions, especially at head ( $O(1)$ ).        | Insert/delete operations may require shifting elements, making them slower ( $O(n)$ ).     |

### 13. TUTORIAL

- Describe an algorithm to **insert an element at the end** of a linked list.
- A company maintains a file of customers, organised as a linked list in alphabetical order of customer names.
  - Describe, with the aid of a diagram, the state of this linked list when the following customer names have been inserted as test data.  
Henry          Nelson          Benny          Sally
  - Explain how
    - Jenny can be added to the list.
    - Henry can be removed from the list.
- Besides searching, inserting and deleting a node from a linked list, describe these algorithms to manipulate linked lists.
  - Count the number of nodes in a linked list.
  - Update a data-item at a particular node in a linked list.
  - Append (or concatenate) one linked list to another.
  - Split one linked list into two lists.

### 14. PROGRAMMING EXERCISE

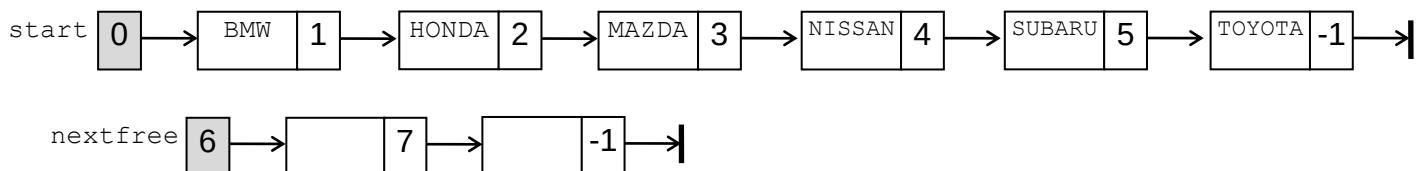
- [implement as arrays]** A programmer implements a linked list of email addresses with pointers **StartPtr**, **NextFree** and **two one-dimensional arrays: Data and Ptr**.

- array **Data** stores the email addresses,
- array **Ptr** stores the link pointers, and
- both arrays have indices of lower bound 1 and upper bound 50.

Code the program to **initialise** both arrays and the procedures to **insert** a new email address to the linked list and **delete** a given email address from the linked list. You may assume no duplicate data is stored in the linked list.

- [implement as array of nodes]** Write program code for a linked list data structure to store a list of car makes.

The diagram shows the linked list after 6 items are added and unused nodes linked together.



- Define a **Node** class that stores the car make and a pointer.
- Define a **LinkedList** class that implements a linked list as an array of 8 nodes, with a **start** pointer that indicates the index of the first node in the data list, and a **nextfree** pointer that indicates the index of the first free node. Using the pseudocode on pages 9 and 10, write methods to **insert** a new item into the linked list and **delete** an item from it. Write also a **display** method to show the linked list in a table form.

(c) Write a **main** program that prints a menu with these options and performs the operations indicated by the options.

1. **Initialise linked list**
2. **Insert item**
3. **Delete item**
4. **Display linked list**
5. **Exit program**

(d) Test your program code by inserting these data in the following order:

**MAZDA, SUBARU, HONDA, TOYOTA, BMW, NISSAN**

Display the linked list after insertion of these data.

(e) Test your program code by deleting these data in order:

**BMW, TOYOTA, NISSAN**

Display the linked list after deletion of these data.

(f) Write additional program code to

(i) search for a data in the linked list and report the index of the node when the data is found or a suitable message when data is not found.

(ii) traverse the linked list and print the data in alphabetical order.

Include these 2 options in the menu and test your program.

3. **[implement as dynamic]** Write a program for the following user-defined types **Node** and **LinkedList** to implement a linked list. Test your program to ensure that the methods work.

| Class: Node    |                                                                                                           |
|----------------|-----------------------------------------------------------------------------------------------------------|
| Identifier     | Description                                                                                               |
| <b>Data</b>    | Email address                                                                                             |
| <b>Pointer</b> | Points to another Node (object).<br>This is <b>not</b> an integer index as nodes are not stored in array. |

| Class: LinkedList        |                                                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| Identifier               | Description                                                                                                        |
| <b>Start</b>             | Points to the first node object of the linked list                                                                 |
| <b>Methods</b>           |                                                                                                                    |
| <b>Initialise()</b>      | Initialises Start to be NULL (i.e.: None).                                                                         |
| <b>IsEmpty()</b>         | Returns True if linked list is empty. False otherwise.                                                             |
| <b>AddNode(Data)</b>     | Adds a new node to the tail of linked list.<br>Takes Data as input argument.                                       |
| <b>DeleteNode(Email)</b> | Deletes the node with Email as Data attribute from the linked list.                                                |
| <b>CaptureToList()</b>   | Traverse nodes in linked list and store the emails of each node into a Python list.<br>Return the list of Strings. |